Computer Programming Assignment No:2

Submitted By:
Abdul Wahab
Electrical Ai
Semester: 2nd
Section: B
Reg No:
BF25NWELE0704

Submitted to:
Engr Muhammad Rizwan
Course:
EE170Computer Programming

University of Engineering & Technology Peshawar (Nowshera Campus)

1. Write a program to:

- **Take 10 integer inputs in an array.**
- **Find and display: Maximum value Minimum value Average**

```
#include <iostream>                          // For input/output
using namespace std;                         // To use cout and cin without std::
int main() {                                 // Main function starts
int arr[10];                                 // Declare array of size 10
int max, min;                                // Variables to store maximum and minimum
int sum = 0;                                 // Variable to store sum (initialized to 0)
cout << "Enter 10 numbers:\n";               // Ask user for input
for(int i = 0; i < 10; i++) {                // Loop to take 10 inputs
  cin >> arr[i];                             // Store input in array
  sum += arr[i];                             // Add value to sum
}
max = min = arr[0];                          // Assume first element is max and min

for(int i = 1; i < 10; i++) {                // Loop from second element
   if(arr[i] > max)                          // Check for maximum
       max = arr[i];                         // Update max
    if(arr[i] < min)                         // Check for minimum
      min = arr[i];                          // Update min
    }
cout << "Maximum: " << max << endl:          // Print max
cout << "Minimum: " << min << endl;          // Print min
cout << "Average: " << sum / 10.0 << endl;   // Print average

    return 0                                 // End program
}
```

2. Write a program that:

- **Uses a function to reverse a 1D array.**
- **Display the array before and after reversal.**

```
#include <iostream>                          // Library for input/output
using namespace std;                         // Standard namespace

// Function to reverse array
void reverseArr(int arr[], int size) {       // Function with array and size
    for(int i = 0; i < size / 2; i++) {      // Loop till half array
        int temp = arr[i];                   // Store first element
        arr[i] = arr[size - 1 - i];          // Replace with last element
        arr[size - 1 - i] = temp;            // Put stored value at end
    }
}
int main() {
    int arr[5];                              // Declare array of size 5
    cout << "Enter 5 elements:\n";           // Ask for inpu
    for(int i = 0; i < 5; i++)               // Loop for input
        cin >> arr[i];                       // Store value
    cout << "Before: ";                      // Print before reversing
    for(int i = 0; i < 5; i++)
        cout << arr[i] << " ";
    reverseArr(arr, 5);                      // Call function
    cout << "\nAfter: ";                     // Print after reversing
    for(int i = 0; i < 5; i++)
        cout << arr[i] << " ";
    return 0;                                // End program
}
```

3. Write a program to:

- **Input two 3×3matrices.**
- **Perform matrix addition.**
- **Display the result matrix.**

```
#include <iostream>                          // Input/output library
using namespace std;
int main() {
    int A[3][3], B[3][3], C[3][3];           // Declare 3 matrices
    cout << "Enter Matrix A:\n";             // Input matrix A
    for(int i = 0; i < 3; i++)               // Row loop
        for(int j = 0; j < 3; j++)           // Column loop
            cin >> A[i][j];                  // Store values
    cout << "Enter Matrix B:\n";             // Input matrix B
    for(int i = 0; i < 3; i++)
        for(int j = 0; j < 3; j++)
            cin >> B[i][j];
    for(int i = 0; i < 3; i++)               // Loop for addition
        for(int j = 0; j < 3; j++)
            C[i][j] = A[i][j] + B[i][j];     // Add elements
    cout << "Result Matrix:\n";              // Display result
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++)
            cout << C[i][j] << " ";          // Print element
        cout << endl;                        // New line after row
    }
    return 0;                                // End program
}
```

4. Write a program that:

- **Stores names of 5 students in a string array.**
- **Sorts them in alphabetical order.**
- **Displays the sorted list.**

```
#include <iostream>                    // Input/output
using namespace std;
int main() {
    string names[5];                   // Array of 5 strings
    cout << "Enter 5 names:\n";        // Input names
    for(int i = 0; i < 5; i++)
        cin >> names[i];
    for(int i = 0; i < 5; i++)         // Outer loop
        for(int j = i + 1; j < 5; j++) // Inner loop
            if(names[i] > names[j])    // Compare strings
                swap(names[i], names[j]); // Swa
    cout << "Sorted Names:\n";         // Display result
    for(int i = 0; i < 5; i++)
        cout << names[i] << endl;
    return 0;
}
```

5. Power System Load Analysis (1D Array + Function) In a power system, hourly load demand (in MW) is recorded. Write a program to:

- **Store load data for 24 hours in an array.**
- **Create functions to calculate:**

  Total load

  Peak load

```
#include <iostream>
using namespace std;
int totalLoad(int arr[]) {                    // Function to calculate total
   int sum = 0;
   for(int i = 0; i < 24; i++)                // Loop through 24 hours
      sum += arr[i];                          // Add values
   return sum;                                // Return total
}
int peakLoad(int arr[]) {                     // Function for max value
   int max = arr[0];
   for(int i = 1; i < 24; i++)
      if(arr[i] > max)
         max = arr[i];
   return max;
}
int main() {
   int load[24];                              // Array for 24 hours
  cout << "Enter 24 load values:\n";
   for(int i = 0; i < 24; i++)
      cin >> load[i];
   cout << "Total Load: " << totalLoad(load) << endl;
   cout << "Peak Load: " << peakLoad(load) << endl;
   return 0;}
```

6. Design a program to: Store student names (string array) and their marks (1D array).

Use functions to:

- **Find topper**
- **Display pass/fail status**
- **Search a student by name**

```
#include <iostream>
using namespace std;
void topper(string name[], int marks[]) {                    // Find topper
   int max = marks[0], index = 0;
   for(int i = 1; i < 5; i++)
      if(marks[i] > max) {
         max = marks[i];
         index = i;}
   cout << "Topper: " << name[index] << endl;}
void passFail(string name[], int marks[]) {                  // Pass/Fail
   for(int i = 0; i < 5; i++)
      cout << name[i] << " "
         << (marks[i] >= 50 ? "Pass" : "Fail") << endl;}
void search(string name[], int marks[], string key) {        // Search
   for(int i = 0; i < 5; i++)
      if(name[i] == key)
         cout << "Marks: " << marks[i] << endl;}
int main() {
   string name[5];      int marks[5];
   for(int i = 0; i < 5; i++)
      cin >> name[i] >> marks[i];
   topper(name, marks);
   passFail(name, marks);
   search(name, marks, "Ali");       return 0;}
```

7. Renewable Energy Monitoring System (1D Array + Functions) A solar power plant records the energy generated (in kWh) every hour for one day. Write a C++ program to:

Store energy generation data for 24 hours in a 1D array.

Create functions to:

- **Calculate total energy generated in a day**
- **Find the hour with maximum generation**
- **Calculate average energy generation**
- **Display all results clearly.**

```
#include <iostream>
using namespace std;
float total(int arr[]) {                    // Total energy
    float sum = 0;
    for(int i = 0; i < 24; i++)
        sum += arr[i];
    return sum;
}
int maxHour(int arr[]) {                    // Max hour
    int max = arr[0], index = 0;
    for(int i = 1; i < 24; i++)
        if(arr[i] > max) {
            max = arr[i];
            index = i;
        }
    return index;
}
int main() {
    int energy[24];
    for(int i = 0; i < 24; i++)
        cin >> energy[i];

    cout << "Total: " << total(energy) << endl;
    cout << "Max Hour: " << maxHour(energy) << endl;
    cout << "Average: " << total(energy)/24 << endl;
    return 0;
}
```

8. Battery Health Monitoring (1D Array + Functions) In battery management systems, voltage readings are monitored to ensure safe operation. Write a program to:

- **Store 20 voltage readings of a battery in an array.**
- **Define safe voltage limits (e.g., 3.0V to 4.2V).**
- **Create functions to:**
- **Count how many readings are below or above safe limits**
- **Find the maximum and minimum voltage**
- **Display a warning message if unsafe readings exist**

```
#include <iostream>
using namespace std;
int main() {
    float v[20];                          // Voltage array
    int low = 0, high = 0;                // Counters
    float max, min;
    for(int i = 0; i < 20; i++) {
        cin >> v[i];
        if(i == 0) {                      // Initialize max and min
            max = min = v[i];
        }
        if(v[i] < 3.0) low++;             // Below safe limit
        if(v[i] > 4.2) high++;            // Above safe limit
        if(v[i] > max) max = v[i];
        if(v[i] < min) min = v[i];
    }
    cout << "Low: " << low << " High: " << high << endl;
    cout << "Max: " << max << " Min: " << min << endl;
    if(low > 0 || high > 0)
        cout << "Warning: Unsafe readings!\n";
```

```
    return 0;
}
```

9. Smart Grid Load Distribution (2D Array + Functions) In a smart grid, electricity consumption is recorded across multiple regions. Write a program to:

- **Store power consumption (in MW) for 4 regions over 7 days using a 2D array.**
- **Create functions to:**
- **Calculate total consumption for each region**
- **Identify the day with highest total demand**
- **Compute overall average consumption**

```
#include <iostream>
using namespace std;
int main() {
    int grid[4][7];                          // 4 regions, 7 days
    for(int i = 0; i < 4; i++)               // Input data
        for(int j = 0; j < 7; j++)
            cin >> grid[i][j];
    for(int i = 0; i < 4; i++) {             // Total per region
        int sum = 0;
        for(int j = 0; j < 7; j++)
            sum += grid[i][j];
        cout << "Region " << i+1 << ": " << sum << endl;
    }
    int maxDay = 0, maxSum = 0;

    for(int j = 0; j < 7; j++) {             // Check each day
        int sum = 0;
        for(int i = 0; i < 4; i++)
            sum += grid[i][j];
        if(sum > maxSum) {
            maxSum = sum;
            maxDay = j;
```

```
        }
    }
    cout << "Highest demand day: " << maxDay + 1 << endl;
    int total = 0;
    for(int i = 0; i < 4; i++)
        for(int j = 0; j < 7; j++)
            total += grid[i][j];
    cout << "Average: " << total / 28.0 << endl;
    return 0;
}
```

10. Classroom Attendance System (2D Array + Functions) Teachers often track student attendance daily. Write a program to:

- **Store attendance for 5 students over 7 days using a 2D array (Use 1 for present, 0 for absent).**
- **Create functions to:**
- **Calculate total attendance of each student**
- **Find the student with highest attendance**
- **Calculate overall class attendance percentage**

```
#include <iostream>                          // Library for input and output
using namespace std;                         // Use standard namespace
// Function to calculate total attendance of each student
void totalAttendance(int att[][7])           // 2D array with 7 columns (days)
{
    for(int i = 0; i < 5; i++)               // Loop through 5 students (rows)
    {
        int total = 0;                       // Initialize total for each student
        for(int j = 0; j < 7; j++)           // Loop through 7 days (columns)
```

```
        {
            total += att[i][j];                                  // Add attendance (0 or 1)
        }
        // Display total attendance of each student
        cout << "Student " << i + 1
             << " total attendance: " << total << endl;
    }
}
// Function to find student with highest attendance
void highestAttendance(int att[][7])
{
    int max = -1;                                                // Variable to store maximum
attendance
    int student = -1;                                            // Variable to store student index
    for(int i = 0; i < 5; i++)                                   // Loop through students
    {
        int total = 0;                                           // Reset total for each student
        for(int j = 0; j < 7; j++)                               // Loop through days
        {
            total += att[i][j];                                  // Calculate total attendance
        }
        if(total > max)                                          // Check if current total is greater
        {
            max = total;                                             // Update maximum
            student = i;                                             // Store student index
        }}
    // Display result
    cout << "Student with highest attendance: Student "
         << student + 1 << " (" << max << " days)" << endl;
  }
```

```
// Function to calculate overall class attendance percentage
void overallAttendance(int att[][7])
{
    int total = 0;                                  // Variable to store total attendance
    for(int i = 0; i < 5; i++)                      // Loop through students
    {
        for(int j = 0; j < 7; j++)                  // Loop through days
        {
            total += att[i][j];                     // Add all attendance values
        }
    }
    // Total possible attendance = students × days = 5 × 7 = 35
    float percentage = (total / 35.0) * 100;        // Calculate percentage
    cout << "Overall class attendance: "
         << percentage << "%" << endl;
}
int main()                                          // Main function starts
{
    int att[5][7];                                  // 2D array for 5 students & 7 days

    cout << "Enter attendance (0 = Absent, 1 = Present):\n";

    // Input attendance data
    for(int i = 0; i < 5; i++)                      // Loop for students
    {
        for(int j = 0; j < 7; j++)                  // Loop for days
        {
            cin >> att[i][j];                       // Take input

            // Validate input (must be 0 or 1)
```

```
            if(att[i][j] != 0 && att[i][j] != 1)
            {
                cout << "Invalid input! Enter 0 or 1 again:\n";
                j--;                                    // Repeat same position
            }
        }
    }
    cout << "\n--- Results ---\n";

    totalAttendance(att);                               // Call function for each student total
    highestAttendance(att);                             // Call function for highest attendance
    overallAttendance(att);                             // Call function for overall percentage
    return 0;                                           // End program
}
```